C++ Strings

- std::string is a Class in the Standard Template Library
  - #include <string>
  - std namespace
  - contiguous in memory
  - dynamic size
  - work with input and output streams
  - lots of useful member functions
  - our familiar operators can be used (+, = , < , <=, >, >=, +=, ==, !=, []...)
  - can be easily converted to C-style Strings if needed
  - safer

Hello this is c++ strings, learn c++ with ease, Interest and u will definitely achieve it one day, all the best and thank you so much, have a nice day byebye

C++ Strings

Declaring and initializing

```
#include <string>
using namespace std;

string s1;                  // Empty
string s2 {"Frank"};        // Frank
string s3 {s2};             // Frank
string s4 {"Frank", 3};     // Fra
string s5 {s3, 0, 2};       // Fr
string s6 (3, 'X');         // XXX
```

C++ Strings

Assignment =

```
string s1;
s1 = "C++ Rocks!";

string s2 {"Hello"};
s2 = s1;
```

C++ Strings

Concatenation

```
string part1 {"C++"};
string part2 {"is a powerful"};

string sentence;

sentence = part1 + " " + part2 + " language";
        // C++ is a powerful language

sentence = "C++" + " is powerful"; // Illegal
```

C++ Strings

Accessing characters [] and at() method

```
string s1;
string s2 {"Frank"};

cout << s2[0] << endl;    // F
cout << s2.at(0) << endl; // F

s2[0] = 'f';      // frank
s2.at(0) = 'p';   // prank
```

C++ Strings

Comparing

`==` `!=` `>` `>=` `<` `<=`

The objects are compared character by character lexically.

Can compare:

- two `std::string` objects
- `std::string` object and C-syle string literal
- `std::string` object and C-style string variable

C++ Strings

Comparing

```
string s1 {"Apple"};
string s2 {"Banana"};
string s3 {"Kiwi"};
string s4 {"apple"};
string s5 {s1};             // Apple

s1 == s5            // True
s1 == s2            // False
s1 != s2            // True
s1 < s2             // True
s2 > s1             // True
s4 < s5             // False
s1 == "Apple";      // True
```

C++ Strings

Substrings – `substr()`

Extracts a substring from a std::string

```
object.substr(start_index, length)
```

```
string s1 {"This is a test"};

cout << s1.substr(0,4);// This
cout << s1.substr(5,2);// is
cout << s1.substr(10,4);   // test
```

C++ Strings

Removing characters – `erase()` and `clear()`

Removes a substring of characters from a std::string

```
object.erase(start_index, length)
```

```
string s1 {"This is a test"};

cout << s1.erase(0,5); // is a test
cout << s1.erase(5,4); // is a
s1.clear();                  // empties string s1
```

C++ Strings

Other useful methods

```
string s1 {"Frank"};

cout << s1.length() << endl;  // 5

s1 += " James";
cout << s1 << endl;          // Frank James
```

Many more…

C++ Strings

Input >> and `getline()`

Reading std::string from cin

```
string s1;
cin >> s1;           // Hello there
                     // Only accepts up to the first space
cout << s1 << endl;     // Hello

getline(cin, s1);  // Read entire line until \n
cout << s1 << end;      // Hello there

getline(cin, s1, 'x');  // this isx
cout << s1 << endl;     // this is
```